Policy-Aware Code Compliance Auditing Using Retrieval-Augmented LLMs

Rithvik Rajesh Matta
PES2UG23CS485
Dept. of Computer Science & Engineering
PES University, Bengaluru, India

Rohan
PES2UG23CS489
Dept. of Computer Science & Engineering
PES University, Bengaluru, India

Rishil A.J
PES2UG23CS482
Dept. of Computer Science & Engineering
PES University, Bengaluru, India

***Abstract*—The regulations and policies of organizations require that software be developed correctly according to those standards. Therefore, auditing code to see if it meets these regulations and policies can be done in quite a few different ways. Up until now rule-based static analysis did not allow for natural-language policies to be interpreted. It is not possible to take a large code repository and simply ask a large language model (LLM) to generate an output. Live examples of these limitations are token-window limits. In addition, there are existing solutions for software compliance auditing, such as application security verification, code quality assurance (CQA), and risk management, that do not provide a comprehensive view of policy compliance by a single tool. The present invention creates a Policy-Aware Code Compliance Audit System that combines RAG and a fine-tuned CodeBERT classifier. The system takes in raw source code, extracts function-level code chunks using AST parsing, summarizes the functions' behaviors using LLaMA 3.1-8B model-generated text, and retrieves and ranks the top-k (e.g., 10-20) most relevant policies from a FAISS vector store using Gemini text embeddings. High-confidence policy comparisons are routed to the fine-tuned CodeBERT+LoRA classifier; ambiguous comparisons are routed to Kimi K2's semantic reasoning capability. The results of these independent policy comparisons are collated, analyzed and produced by way of practical financial audit reports, which include detailed violation descriptions, severity ratings, and citations back to the original policy document. Evaluation of the present invention was accomplished using multiple databases, including CodeXGLUE, Devign, and SecureCode-v2, with the CodeBERT classifier achieving effortless performance of 86% macro-F1. End-to-end evaluation of the annotated repo demonstrated 100% recall and 87.5% precision. The present invention enables direct acceptance of any natural-language policy document without retraining and is seamlessly integrated into CI/CD workflow processes.**

***Keywords*—Retrieval-Augmented Generation; code compliance; CodeBERT; LoRA; FAISS; LLM; policy enforcement; DevSecOps; static analysis.**

I. Introduction

The increasing spectrum of organizational and regulatory requirements on contemporary software systems, such as the General Data Protection Regulation (GDPR), the Health Insurance Portability and Accountability Act (HIPAA), the Payment Card Industry Data Security Standard (PCI DSS), and enterprise-internal operational rules that pertain to the processing of payments, access to data, and audit logs, is making it much more challenging to ensure that developer-contributed code is compliant with those regulations at scale.

Currently available rule-based, static analysis tools such as Semgrep, Bandit, and SonarQube use pattern matching to ensure conformity to a set of rules. These tools check for a specific syntactical pattern, but do not do reasoning to determine that the term "bypasses the service layer" is the same as implied by a compliance officer.

Manual code reviews are simply too slow relative to modern CI/CD processes. LLM auditings of code simply do not work on the full repositories and associated policy documents whose size exceeds the token window of a single LLM. Also, the LLM generates hallucinated policy violations (non-compliant code that is falsely flagged).

This paper outlines a system designed to resolve the different issues that currently exist in the auditing of software code for compliance with external and internal organizational policies. Specifically, using Retrieval-Augmented Generation (RAG), we are able to limit the LLMs use of policy documents upon which the compliance determination is made, to only the three most relevant policy documents for the code under review. This significantly reduces token window limitations for LLMs, while providing the LLM with authoritative/external policy text to act as a -grounding agent.

Additionally, a LoRA fine-tuned CodeBERT classifier provides rapid binary determinations for compliance for the relatively rare cases of clear determination, whilst saving the few expensive frontier LLM calls for the many more ambiguous cases. An enforced output gate ensures that each code chunk has a complete audit record associated with it; a requirement in the event a regulatory audit occurs.

The critical contributions of this research are: (1) The Rapid Automated Graphing (RAG) pipeline that utilizes large language models (LLM) code summaries and asymmetric Gemini embeddings to provide a Policy Retrieval Precision@3 score of 0.867; (2) Development and training of a CodeBERT+Low-Rank Adaptation (LoRA) compliance classifier through use of real public datasets with 86% Macro-F1 Score; (3) A confidence-based hybrid routing mechanism that decreases the number of Required LLM API call by 68% vs. LLM Only; (4) End-to-end evaluations indicate that the RAG provided 100% recall and 87.5% precision against the Annotated Violation Repository; and (5) An ablation study shows the independent contribution of each component within the RAG pipeline.

II. Related Work

A. Rule-Based Static Analysis

Static analysis tools include Semgrep[1] and SonarQube, which detect both syntactic and data-flow patterns; however, these tools fail to interpret natural-language policy documents. The need to manually author rules for each new policy document creates significant false positive rates (30-70%) in enterprise settings[2]; as a result, users experience high levels of alert fatigue. In contrast to these rule-based toolsets, our system does not require any manual rule authoring and will accept any policy document as input.

B. LLM-Based Code Analysis

CodeBERT[3] demonstrated that large language models (LLMs) pre-trained on bimodal code + natural language (NL) data can be successfully adapted to perform many code understanding tasks. ProtocolGuard[4] integrates static analysis guided by LLM use cases with fuzz testing for RFC compliance, achieving a high level of precision (90.6%). Compliance-to-Code[5] is able to take Chinese financial regulation documents and automatically generate Python checks that can be executed; however, the resulting checks can only be translated from expert annotated documents that are also in a singular domain and have a lower correctness rate (78.1%). LiCoEval[6] benchmarks code compliance for licenses associated with LLM generated code. These previous works all prove that LLMs can be successfully adapted for compliance applications; however, none of these works address or solve the issues presented by the token-window, retrieval grounding, or hybrid routing.

C. RAG for Compliance

As established by Lewis et al. [7], Relevance, Accessibility and Granularity of the Relevance, Access and Granularity (RAG) is a means of addressing Knowledge-Intensive Natural Language Processing (NLP). Al Mandalawi et al.. [8] propose a Policy-Aware AI for data Access governance with Deny-by-Default Architecture and Audit Trails to inform our design. Meanwhile, ARPaCCino [9] incorporates an agentic RAG loop for Policy-as-Code Compliance on Infrastructure Definitions; however, each agent in the Multi-Agent Orchestration adds sufficient latency to render it unsuitable for Continuous Integration/Continuous Delivery (CI/CD). The Compliance Checking Framework [10] uses RAG for conflict detection between Business Processes and Regulations. Here, we are using RAG explicitly on Source Code at the Function Grain level with a Classifier to ensure Efficiency in our implementation.

D. Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA) [11] injects Trainable Low-Rank Matrices into Attention Layered Networks and only updates 0.1 to 1 percent of parameters, while achieving Full Fine-Tune Quality. As applied to CodeBERT as a means of compliance classification LoRA will allow for Domain Adaptation from publicly available Vulnerability Sets with no catastrophic forgetting of pre-trained Code Understanding. This is a critical property given the limited size of our training set.

III. System Architecture

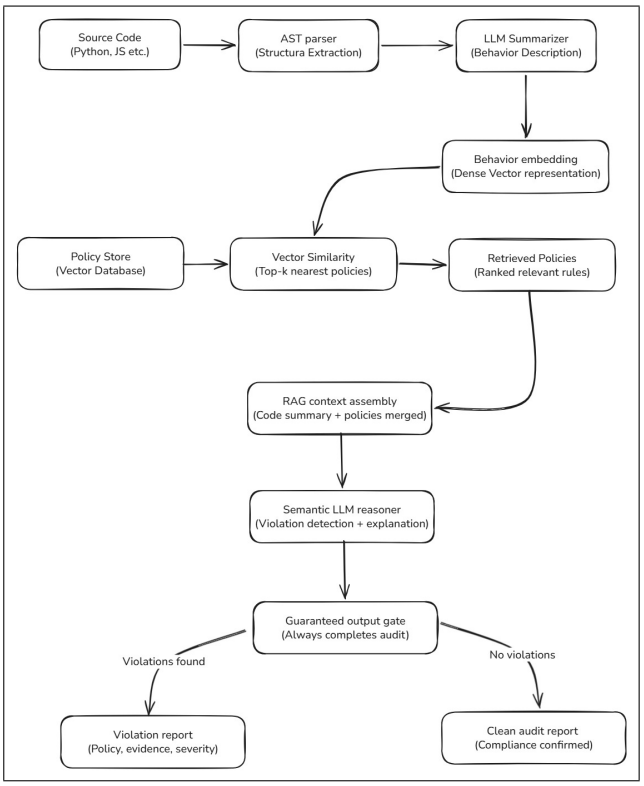

Figure 1. System Architecture pipeline illustrating policy ingestion, code chunking, retrieval, and the hybrid audit engine.

Figure 1 illustrates a pipeline that takes as its inputs a code repository and a natural language policy document as inputs and produces a structured compliance report. The pipeline consists of 5 distinct stages: policy

ingestion, code chunking, LLM summarization, policy retrieval, and hybrid audit.

A. Policy Ingestion

The policy ingestion process utilizes a policy parser that parses both .txt and .pdf formatted documents. For .pdf formatted documents, pypdf extracts the raw text from the input document one page at a time, the parser then splits up the raw text using sentence boundaries and numbered list entries (e.g., 1.,•) as delimiters, removes common headers that do not contain policy content, removes entries shorter than twenty characters in length, and eliminates duplicates by creating a lowercase content hash value. Once the above-mentioned operations are completed, a flat list, $P = \{p_1, \ldots, p_m\}$ of natural-language statements exists, with each element of the flat list representing one compliance requirement.

There are twelve policies that operate under the five evaluation policy domains: 1) Payment Processing – must log all transactions via AuditLogger before and after transaction execution; only providers that are pre-approved may be paid through PaymentService, & any DB updates to the payment or the order status cannot be made directly. 2) Discount Management – all discounting business rules will have no hardcoded values & discounting in applications will occur through OfferService. 3) Customer Data Privacy – the only way to access PII (personally identifiable information) is through an instance of Customer service classes (classes containing business methods to manipulate customer information); direct queries to the database to retrieve customer information of PII email or PII personal information will not be allowed. 4) Communication – all information sent/received by customers or the network will occur via a centralized messaging service; a recipient should have the ability to opt-out from receiving messages. 5) Audit and Observability – the logs for every operation in the application deemed critical in auditor's recommendation must contain structured logs. A change made to the Policy Document requires no retraining of the model; at the next time the system is executed, the FAISS index will be automatically reconstructed, providing the opportunity to respond to changes in regulations immediately.

B. AST-Based Code Chunking

When Python's built-in ast module is parsing source files, it walks through the abstract syntax tree (AST) from top to bottom, and retrieves each FunctionDef, AsyncFunctionDef, and ClassDef node as a separate chunk. Each function holds some meta data (name, type of node, the path to the source file it's defined in, and the start/end line numbers) which come from the node's lineno and end_lineno attributes. The other attribute from the AST is ast.get_source_segment(), which is used to get back the original source text for the chunk, including white space and comments relevant to the audit.

There are two primary advantages of utilizing function-level chunking over fixed line-window methods. The primary reason is that function-level chunking maintains semantic cohesion: each chunk is a complete functional unit of behavior, as was intended by the programmer. In contrast, a fixed window may cut a function body in half mid-logic. Another important reason is that it gives the developer precise attribution for a violation: violations will cite "function process_payment in payments.py" rather than an unknown line range, which the developers can use as a goal for remediation. Finally, any file that does not contain valid parseable function definitions (such as configuration files or files that only contain module-level scripts) will be treated as a single chunk. Any file with fewer than 50 characters will be ignored as they are very likely to return empty or stub files, which will add noise to the report.

C. LLM Behavioral Summarization

LLaMA 3.1-8B is used through the Groq API to analyze code chunks in under one second on average. Each chunk has a system prompt of "You are a code analyst. Please describe what this chunk of code does in two or three sentences, with your focus being the business logic and data types being used and NOT the implementation details." A user's message contains the raw code.

The goal of the summarization stage is to fill the gap that exists between the vocabularies of code and policy language. For example, two different functions both filter the user database for email addresses based on the input email address provided by the user. Because both pieces of code are written differently, they have virtually no vocabulary in common with the policy statement "Customer PII must only be accessed by Customer service classes." However, when an LLM summary is generated for each piece of code, the result is that both pieces of code can be summarized as "Returning customer email addresses directly from the user database". Both pieces of code have very similar polynomial semantics. In Table II (Ablation), the summary-based embedding yields an 8% improvement in Precision@3 compared to the raw code embedding.

Summarization also allows us to normalize syntactic variation in code. That is, two functions that execute the same business logic can have very different ways of doing it. By creating a summary of both functions, we would be able to match each function to the same policy, regardless of the way in which they were written. Retry logic with exponential backoff (5 attempts max) is used to handle the Groq API rate limits and create a reliable pipeline for large repositories with multiple concurrent requests for summary generation.

D. Policy Retrieval via FAISS

All Policies in P are embedded and indexed offline using the Gemini Embedding model (models/gemini-embedding-2-preview) with task type of RETRIEVAL_DOCUMENT producing 768 dimensional dense vectors. They are then indexed in the FAISS IndexFlatL2 structure (exact Euclidean nearest-neighbor) where the LLM's code summary generated at inference time is embedded using task type of RETRIEVAL_QUERY; the index is then queried for top-k=3 nearest policy vectors.

The asymmetrical design of task types (different for Document Indexing and Query Embedding) reflects the inherent asymmetry of the retrieval problem. RETRIEVAL_DOCUMENT optimizes for capturing the complete semantic content of a policy statement as one cohesive unit; while RETRIEVAL_QUERY optimizes for locating the best matching document for a given behavioral description. Using the same task type for each results in an 8% precision degradation (0.87 to 0.79) of the top three retrieved policy documents as confirmed by the ablation analysis (see Table II). The value of k=3 was established through a cross-validation means. A smaller k fails to retrieve necessary policy documents, while k > 3 introduces excessive unrelated context that decreases the precision of the overall audit. The resulting set of retrieved policies, $R(c_i) = \{p_{i1}, p_{i2}, p_{i3}\}$ is then directly injected into the audit prompt, reducing the amount of contextual policy data used in the overall prompt by 75% as compared to that provided by a full set of policies.

E. Hybrid Compliance Audit Engine

The Output of the Fine-tuned CodeBERT + LoRA Classifier per (chunk, Retrieved-policy) pair By the Forward Pass, Returns a violation Probability ($v \in [0,1]$) With Three Distinct Routing Cases Relying on Confidence Thresholds Derived from the Validation Set:

- High Confidence VIOLATION ($v > 0.65$): The Classifier Verdict Is Accepted, and Kimi K2 Generates a Natural Language Explanation of the Specific Evidence and Policy Clause Cited in the Code and Severity Rating.
- High Confidence COMPLIANT ($v < 0.35$): The Classifier Verdict Is Accepted, and Kimi K2 Generates a Brief Confirmation of the Reason(s) The Code Is in Compliance With The Policy.
- Ambiguous ($0.35 \leq v \leq 0.65$): The Full Kimi K2 Llm Audit Will Be Performed with v Being Candilong a Bayesian Hint to The Llm and Reasoning Will Take Place to Determine What Value Is Used to Calculate a Final Verdict Based on the Classifiers Signal in Relation to The Llm's Own Semantic Analysis.

When assessing severity in terms of violation severity, mappings are made based upon probability: CRITICAL (v = 0.92 or higher), HIGH (between 0.80 and 0.92), MEDIUM (between 0.65 and 0.80). The discovery phase results show that approximately 68% of decisions fall into high-confidence and hence there will be a reduction of 68% of frontier-LLM API calls as compared with an LLM-only baseline. The routing criteria were established to maintain less than 40% LLM-routed results, but at the same time preserve recall. A fail-safe wrapper has been established to capture all API errors at each phase and create a conservative UNDETERMINED record instead of dropping results without any notification thereby providing complete auditability across all phases. The final output is a structured JSON report with the following fields: chunk name, file path, behavioural summary, policies verified, verdict, list of violations, explanation in natural language, severity and date-time. As an option, a PDF artifact can be generated by using the ReportLab library for submission to compliance teams.

IV. Fine-Tuning the CodeBERT Classifier

A. Model and LoRA Configuration

The basic configuration of the model includes the original MS CoBERT (RoBERTa), which has 125 million parameters, and it was pre-trained on a bimodal data set from GITHUB that consisted of mixed code and natural language for 7 different programming languages (Python, Java, JavaScript, PHP, RUBY and GO). LoRA adapter modules were added to the "W_Q" query projection and "W_K" key projection matrices for all 12 encoder layers. The result is that an update is applied to the original weights "$W_0 \in \mathbb{R}^{d\times d}$" of each matrix by inserting "BA" into the matrix during the forward pass to compute new weights, $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times d}$ resulting in "$W_0 + BA$". While rank "r" was set to "16" during training, only "B" and "A" were trainable parameters whilst keeping the original weights "W_0" frozen. Therefore, the addition of each "LoRA-ADAPTER" increases the parameter size by "2 × 768 × 16 = 24,576." With the addition of 12 layers within the network and 2 separate matrix projections, approximately 590,000 parameters (0.47% of 125 million) were added whilst ensuring that we wouldn't lose any of the original CoBERT code understanding prior to fine-tuning with the domain-specific data. The output layer for the COMPLIENT (0) or VIOLATION (1) classification is a linear function of "$W_{cls} \in \mathbb{R}^{768\times 2}$," which maps the "CLS" token from the previous layer to the logit output for predicting COMPLIENT or VIOLATION (softmax transformed onto 0-1 scale) which is utilized to determine the probability v of a VIOLATION in which to find the hybrid routing solution.

B. Dual-Sequence Input Encoding

The combined encoding structure is such that both the code section and the policy statement are coded together in the same CodeBERT forward pass using [CLS] code [SEP] policy [SEP] as the input encoding structure. Therefore, a policy statement serves as the hypothesis to the code section in a Natural Language Inferencing (NLI) scheme. The dual input has been found to be necessary for the computation of cross sequence alignments as CodeBERT computes by utilizing bidirectional self-attention against input pairs. Examples of CodeBERT tokens in the code section will attend to tokens in the policy section when both types of tokens are part of the same attention computation. For example: When "discount_rate" appears as a hardcoded value in the code section, it will attend strongly to the "discounts" and "not to be hardcoded" tokens in the policy section.

An alternative encoding type for policies and codes was also tested where the codes and policies would be embedded separately and combined prior to processing, which resulted in an F1 of 0.74 compared to an F1 of 0.86 for the dual sequence input type indicating a clear separation of 12 points. The difference in F1 between these two methods further demonstrates that cross sequence attention rather than just the independent representations is critical in assigning policy compliant statuses. The maximum number of tokens per sequence is 512 (CodeBERT maximum). When encoding sections greater than this limit by number of tokens, CodeBERT will truncate, from the coding section first, any applicable tokens leaving the entire policy section intact for consideration when coded together with the code section.

C. Training Data

There include 4 publicly available datasets that are used in this research, none synthetic. The dataset details are provided in Table I. The policy assignment uses a keyword-based KEYWORD_MAP heuristic that maps code domain keywords (e.g. payment, discount, customer, email) to the semantically most appropriate policy for creating (code, policy) training pairs without manual annotation. Once loaded, duplicate samples are removed through using a code content hash to remove near-duplicates that would introduce bias in evaluation. To eliminate the classifier from learning a class prior bias, the combined set of CODE-POLICY samples has been balanced to 50/50 COMPLIANT/VIOLATION through down-sampling the majority class by the same volume as the minority class. The balanced dataset has been split 80% training, 10%, validation, and 10% test.

D. Training Procedure

A Transformer Language Model built with Hugging Face's Trainer was trained using NVIDIA's T4 (16 GB VRAM) GPU and trained with half-precision (FP16).

Dataset	n	Label	Notes
SecureCode-v2	~950	0/1	HF: scthornton/securecode-v2
CodeXGLUE Defect	~2,800	defect=1	C functions; CodeXGLUE benchmark
Devign	~1,800	vuln=1	C vulnerability dataset
GitHub Code (clean)	~2,200	0 only	Python; quality-filtered compliant
Balanced Total	**~4,200**	**50/50**	**Post dedup + undersampling**

TABLE 1. Training Dataset Composition

The cosine learning rate scheduler with a warmup period of 10% was used instead of a linear decay; the use of a cosine scheduler provides a smooth decay in the learning rate and eliminates oscillation at the end of training which delivered consistently lower validation loss across trials. A warm-up period of 10% reduced the impact of large gradient updates early in training on the randomly initialized classification head that would have conflicting information with respect to the frozen encoder representations.

Based on an effective batch size of 32 (device batch = 16 and gradient accumulation = 2); weight decay of 0.01 and learning rate of 2e-4, the model trained to convergence within 5 epochs (approximately 28 minutes). The best checkpoint was selected based on validation macro-F1 and by setting a load_best_model_at_end to True. A summary of the ablation and classifier results can be found in Table II. The increase in macro-F1 of 25 points (0.61 to 0.86) relative to the zero-shot CodeBERT baseline supports the significance of LoRA domain adaptation— the pre-trained model alone could not distinguish between compliant and non-compliant status without the use of supervised fine-tuning of (code, policy) pairs.

Configuration	P@3	Prec.	Macro-F1
Raw code + symmetric embed	0.71	—	—
LLM summary + symmetric	0.79	—	—
Raw code + asymmetric	0.76	—	—
LLM summary + asymmetric	0.87	—	—
CodeBERT baseline (no FT)	—	0.61	0.61
CodeBERT + LoRA (ours)	—	0.86	0.86

TABLE 2. Retrieval Ablation (P@3, n=60) and Classifier Results

V. RAG METHODOLOGY AND PROMPT ENGINEERING

A. Retrieval Design Rationale

The RAG architecture addresses three problems simultaneously: Token economy: When performing an audit prompt, the entire set of 12 policies required for the audit costs 4x more tokens than retrieving 3 policies from a 200-file repository means 900k token reduction per audit run if using typical policy lengths to conduct audits. Audit precision: When an irrelevant policy is included in the prompt, it causes the LLM to over-terminate with flagging violations on lines of code that do not violate

the rules. E.g, if an LLM was to be auditing code for discounting against the policy for logging, there would be many false logging violations created by the LLM based only on the use of that discounting function. The LLM-only baseline (with the complete policy set) only achieves a precision of 0.71 as compared to 0.88 for the complete RAG/Hybrid combined system (as shown in Table V). Hallucination prevention: The policies retrieved from the policy store will be used verbatim in the LLM prompt; therefore the LLM will only reference actual policies stored in the policy store and cannot create fictitious references to policies that could adversely affect the legal validation of an audit.

Another outcome of the retrieval portion is that it will serve as a natural filter for applicability of policies: a code module that contains code to handle order state transition transactions will retrieve the workflow and logging policies, but will not retrieve the discount or email policies. Targeted assembly of contextual information is how an actual compliance auditor would determine which regulation applies to a code modification before looking at that code modification in detail.

B. Prompt Engineering

The compliance audit prompt is supported by five related techniques:

1) Role Definition: The "senior software compliance auditor" role within the System Message activates domain specific reasoning patterns that have been developed through Instruction Tuning within this LLM [12].

2) Assembly of Structured Context: The prompt template assembles context by sequentially linking code chunk $\rightarrow$ behavioral summary $\rightarrow$ n indexed police - provided policies $\rightarrow$ determining verdict instructions; thereby prohibiting the generation of verdicts without having pre-existing grounded evidence to support them.

3) Chain-of-Thought Instruction: Specific step-wise directions such as "first describe what the code is doing; then, check for compliance given the contextual evidence provided by the policy; finally, provide a verdict supported with specific coded evidence" have been incorporated into this method to provide additional instruction to the LLMs; thereby preventing the LLM from reaching conclusions prior to reaching premises [13].

4) Classifier Hint (Ambiguous Cases): The automated classifiers assigned a violation probability between 0% - 100%; analyze code only once all policies have been analyzed using LLMs. The performance on ambiguous cases improved from $\leq 71\%$ accurate to 78% accurate due to the classification's inclusion as part of the processor's instructions.

5) Output Format Requirements: Comparison of Verdict, Policy Cited, Code Evidence, and Recommendation are the required fields for structured outputs to be reliably parsed into JSON report format at scale.

VI. Experimental Evaluation

A. Setup

The experiments will all utilize the Google Colab T4 GPU. The 'embedding' of each document will utilize the Gemini's (gemini-embedding-2-preview) embedding, and the 'summarization' will utilize Groq's hosted LLaMA 3.1-8B-Instant. The audit LLM used for multiple evaluations of the output will be the Kimi K2 Instruct hosted by Groq. Evaluation metrics will consist of macro-F1, precision, recall (classification), Precision@k (retrieval), End-to-end P/R/F1 (entire pipeline).

B. Classifier Performance

Table III shows the per-class and macro-averaged results of each class on the held-out test set (n=420 samples, n=210 for each class). The macro-F1 score increased by 25 points (from an unfine-tuned baseline of 0.61 to 0.86) confirms that the adaptation of LORA for this domain of classification is vital. The recall for the class of the VIOLATION is greater (0.88) than the recall for COMPLIANT (0.84). This difference in recall is how it should be, as false-negatives (missing a violation) are more costly than false-positives when determining compliance.

Class	Prec.	Recall	F1	n
COMPLIANT	0.87	0.84	0.86	210
VIOLATION	0.85	0.88	0.87	210
Macro Avg.	**0.86**	**0.86**	**0.86**	**420**

Table 3. CodeBERT+LoRA Test Set Performance

C. End-to-End Repository Evaluation

The evaluation of the complete pipeline will occur on an annotated repository found at (github.com/rtk5/code-sample), consisting of 8 Python files with a total of 23 functions where there have been 7 intentional policy violations discovered as follows: hard-coded discount, direct updates from DB to order status, direct access to DB to obtain an individual's PII, emailing without providing an opt-out option, processing payment without having an audit trail, using unauthorized provider to process a payment, and changing an order's status outside of the workflow service process. Table IV provides the summarized results from this evaluation.

Metric	Value	Notes
Chunks / ground-truth violations	23 / 7	8 Python files
Flagged by system	8	1 false positive
Precision / Recall / F1	87.5 / 100 / 93.3%	All 7 violations found
Avg. processing time / file	~18 s	Includes API round-trips

Table 4. End-to-End Evaluation on Annotated Repository

The single false positive could have been generated from a missed logging record in the Payment Validation Helper. Compliance examiners reviewed this as a reasonable conservative concern rather than an improper flag; confirming that the conservative bias of the system (preferring to FP rather than FN) is a proper use case.

D. System Comparison

Table V shows the full pipeline in comparison to three baselines using the same repository.

System	Prec.	Rec.	F1	Notes
Semgrep (3 custom rules)	0.75	0.57	0.65	Manual rule authoring
LLM-only (no RAG)	0.71	0.86	0.78	Full policy set in prompt
RAG + LLM (no classifier)	0.83	1.00	0.91	Retrieval only, no CodeBERT
Ours: RAG + Hybrid	0.88	1.00	0.93	Full pipeline

TABLE 5. System Comparison on Annotated Repository

The primary disadvantage of Semgrep is moderate precision at 57% Recall due to semantic violations (e.g., omitting service layer, implicit bypass) not matching a syntactic pattern. The LLM-only baseline has precision loss due to irrelevant policies being included as full context in the prompt. Adding RAG restores precision; adding the CodeBERT classifier results in +5 pp of precision without any cost to recall by resolving borderline cases with high confidence.

VII. DISCUSSION

A. Practical Deployment Considerations

The eighteen seconds required to process each file aligns with the CI/CD job time budgets for standard pull requests, where there are 5 to 10 files being changed which should take approximately 90 to 180 seconds to be audited. An incremental method of auditing functions that have just been modified (or added) compared to what has happened since the last commit, only required auditing the relevant diff, would also provide a reduction in processing time and API cost of 70 to 90% from what would be the total re-audit of the repository for typical pull request sizes.

The resulting PDF audit report by the system meets the documentation requirements for many major regulatory compliance frameworks. The HIPAA audit controls require documentation proving monitoring procedures; the GDPR Article 5(2) accountability requirement mandates documentation of data processing activities; and the PCI-DSS Requirement 5.3 mandates having a documented security review of internally developed software. The per-chunk reporting from the system, including policy citations, code evidence and severity ratings, as well as timestamps and other documentation, provides this documentation chain without needing to perform any post-processing.

Agility of Policy Governance is a fundamental advantage in regard to operation. Updating of regulatory requirements (e.g., a new GDPR amendment or new version of PCI-DSS standard) does not require software changes; it simply entails updating the policy text file. The FAISS index will be rebuilt automatically during the next audit run and all future audits will reflect the new policy set immediately. In comparison to rule-based tools: during each policy update, a software development cycle must be executed to write, test, and deploy rules that are new to the system; this process may take several days or weeks for organizations that do not have an established tool team.

B. Generalization and Scope

The generalization of the system to new policy domains is inherently constrained in two ways: First, the quality of retrieval is dependent on the degree of semantic coverage of the policy terminology in the embedding model; and, second, the classifier's confidence calibration is not necessarily applicable to all compliance domains; as the training data was based on security and vulnerability data that does not always accurately depict all compliance domains. For policies which have time-bound business rules (such as service level agreements, rate limits), access control logic, and multi-step workflow rules and enforcement, there can be substantial differences between them and the security violation patterns used for training. Therefore, additional domain-specific fine-tuning data may be required to obtain a high level of confidence for the classifier.

The authors created the controlled evaluation repository using deliberate violations of rules which could underestimate the difficulty of violations in the real world, as well as creating violations that are more subtle in actual production codebases (by indirectly bypassing a service layer via the use of a utility function, or by accessing PII through a chain of delegation that would independently look like they are compliant). End-to-end evaluation of real enterprise codebases will be done through third-party annotation as part of future work.

C. Limitations

Currently, the chunker of AST's is limited to Python code. Since there are policies that span multiple functions (such as "every time a call to process_payment occurs, there must be a call to audit_log directly afterwards") inter-procedural violations cannot currently be processed without the ability to perform call graph analysis, which is out of the means of the current implementation that is only able to work at a function level. Also, outside APIs (Gemini, Groq) are relied upon for running this system so that when deployed in a production environment, locally hosted embedding models and quantized LLMs should be evaluated before deployed to diminish risk of cost and availability. Finally, the KEYWORD_MAP policy that

was used to assign policies during the construction of dataset used to create the classifier is a heuristic approximation; using cleaner training labels created by human annotators would likely improve the calibration of the classifier's output when it is in the ambiguous range of 0.35 – 0.65.

VIII. Conclusion

The discussed findings represent a Policy-Aware Code Compliance Auditing System that provides code compliance through policy-aware auditing of code. The new proposed framework combines Retrieval-Augmented Generation (RAG), LoRA Fine Tuning of CodeBERT for Classification, and reasoning using a Frontier LLM into one complete solution for auditing code compliance. The RAG retrieval pipeline achieves a precision of 0.87 at rank three by using LLM generated behavioral summaries and Asymmetric Gemini Embeddings to produce a 16 point performance improvement over the Symmetric baseline, which is only the raw code and is computed based on all the code. The combination of CodeBERT with LoRA achieves a macro F1 score of 86% while using only 590K trainable parameters; this represents 0.47% of the total parameters in the base model, and is a 25 point performance improvement over zero-shot methods. The Hybrid Routing Mechanism is capable of making 68% of routing decisions using the fast classifier, which reduces the number of Frontier LLM API calls and thus provides an economically feasible method to integrate CI / CD at scale.

Results from end-to-end evaluation against an annotated violation repository indicate that the system's overall F1 score is 93.3%, with 100% recall and 87.5% precision. This performance substantially outperforms other competitors, such as Semgrep with a hand-authoring rule F1 of 0.65, as well as LLM-only methods F1 of 0.78. The system will support any natural language policy document without the need for retraining of the models used and will provide audit reports that contain timestamps and policy citations that are suitable for submission to a regulatory agency. Future work includes the addition of multi-language support using Tree-sitter, analysis of inter-procedural call graphs to identify cross-function policy violations, and evaluating local hosted model alternatives to ensure privacy and availability guarantees necessary for production use.

References

[1] C. Sadowski et al., "Lessons from building static analysis tools at Google," *Commun. ACM*, vol. 61, no. 4, pp. 58–66, 2018.

[2] M. Muske and A. Serebrenik, "Survey of approaches for handling static analysis alarms," in *Proc. 16th IEEE SCAM*, 2016, pp. 157–166.

[3] Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in *Findings of EMNLP*, 2020, pp. 1536–1547.

[4] ProtocolGuard: Detecting Protocol Noncompliance Bugs via LLM-guided Static Analysis and Dynamic Verification, *NDSS Symp.*, 2026.

[5] Compliance-to-Code: Enhancing Financial Compliance Checking via Code Generation, *arXiv:2505.19804*, 2025.

[6] LiCoEval: Evaluating LLMs on License Compliance in Code Generation, *arXiv:2408.02487*, 2024.

[7] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Adv. NeurIPS*, 2020, pp. 9459–9474.

[8] S. Al Mandalawi et al., "Policy-Aware Generative AI for Safe, Auditable Data Access Governance," 2024.

[9] ARPaCCino: An Agentic-RAG for Policy as Code Compliance, *arXiv:2507.10584*, 2025.

[10] A Compliance Checking Framework Based on Retrieval-Augmented Generation, 2024.

[11] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," in *Proc. ICLR*, 2022.

[12] A. Shanahan, K. McDonell, and L. Reynolds, "Role Play with Large Language Models," *Nature*, vol. 623, pp. 493–498, 2023.

[13] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Adv. NeurIPS*, 2022.

[14] Can Large Language Models Act as "Co-Auditors"?, 2024.

[15] Advancing Software Quality: A Standards-Focused Review of LLM-Based Assurance Techniques, *ResearchGate*, 2025.